

1. Introduction

La centralisation des données sur un serveur unique présente un risque majeur : le point de défaillance unique (SPOF). La Phase 3 vise à transformer notre base de données MongoDB en un **système distribué et tolérant aux pannes**. Nous avons déployé un **Replica Set** (ensemble de réplication) composé de 3 nœuds, permettant la redondance des données et la continuité de service en cas de panne matérielle.

2. Architecture du Cluster (T3.1)

Nous avons configuré une architecture **PSS (Primary - Secondary - Secondary)** sur la machine locale, en simulant 3 serveurs physiques via des ports distincts.

Schéma de l'architecture

- **Primary (Port 27017)** : Reçoit toutes les écritures et les lectures par défaut.
- **Secondary 1 (Port 27018)** : Réplique les données du primaire en temps réel (Oplog).
- **Secondary 2 (Port 27019)** : Idem, assure le quorum pour les élections.

Procédure de Mise en Place

1. **Stockage** : Création de trois répertoires distincts (`data/mongo/db-1` , `db-2` , `db-3`).
 2. **Démarrage** : Lancement de 3 processus `mongod` avec l'argument `--replSet rs0` .
 3. **Initialisation** : Utilisation de la commande `rs.initiate()` via un script Python pour lier les membres entre eux.
 4. **Peuplement** : Ré-exécution des scripts de migration de la Phase 2. Grâce à la réplication, les données insérées sur le port 27017 ont été automatiquement copiées sur les ports 27018 et 27019.
-

3. Tests de Tolérance aux Pannes (T3.2)

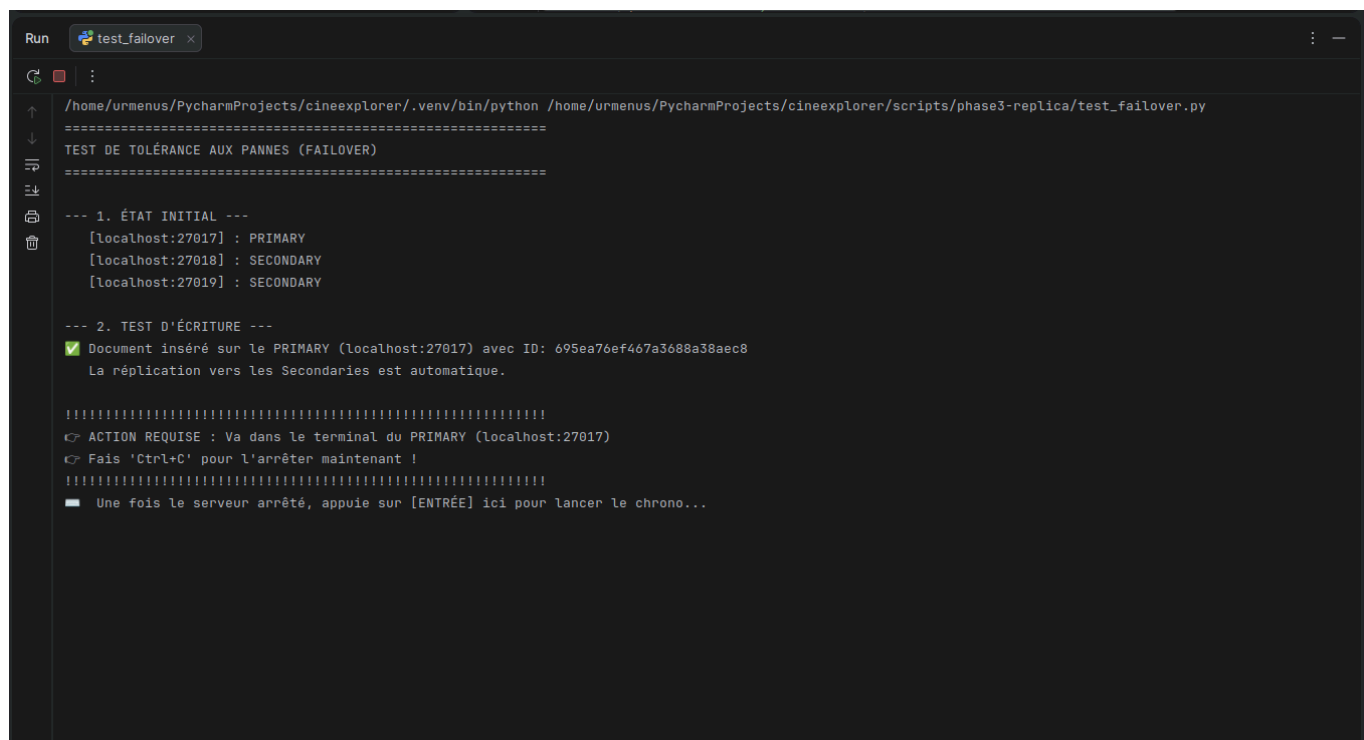
Un script de test interactif (`test_failover.py`) a été développé pour valider la robustesse du cluster. Voici les résultats observés.

Scénario 1 : Panne du Nœud Primaire (Failover)

Action : Arrêt brutal du processus `mongod` correspondant au PRIMARY (Port 27017) via `SIGINT` (Ctrl+C).

Observations :

1. Les nœuds Secondaires ont détecté l'absence de "heartbeat" du Primaire.
2. Une élection a été déclenchée.
3. Le nœud sur le port **27018** (anciennement Secondary) a été élu **NOUVEAU PRIMARY**.



```
Run test_failover x
/home/urmenus/PycharmProjects/cineexplorer/.venv/bin/python /home/urmenus/PycharmProjects/cineexplorer/scripts/phase3-replica/test_failover.py
=====
TEST DE TOLÉRANCE AUX PANNES (FAILOVER)
=====

--- 1. ÉTAT INITIAL ---
[localhost:27017] : PRIMARY
[localhost:27018] : SECONDARY
[localhost:27019] : SECONDARY

--- 2. TEST D'ÉCRITURE ---
✓ Document inséré sur le PRIMARY (localhost:27017) avec ID: 695ea76ef467a3688a38aec8
La répllication vers les Secondaries est automatique.

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
ACTION REQUISE : Va dans le terminal du PRIMARY (localhost:27017)
Fais 'Ctrl+C' pour l'arrêter maintenant !
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
■ Une fois le serveur arrêté, appuie sur [ENTRÉE] ici pour lancer le chrono...
```

```
Run test_failover x
Recherche du nouveau Primary...
✓ NOUVEAU PRIMARY DÉTECTÉ : localhost:27018
⌚ Temps de bascule (Failover) : 0.00 secondes
🔥 L'ancien Primary (localhost:27017) est mort, vive le nouveau (localhost:27018) !

--- 5. VÉRIFICATION DES DONNÉES ---
✓ DONNÉES PRÉSERVÉES ! 1 document(s) retrouvé(s).
Cela prouve que la donnée a eu le temps d'être répliquée avant le crash.

=====
🔧 ACTION REQUISE : Relance le nœud que tu as arrêté (localhost:27017)
Commande rappel : mongod --replSet rs0 --port <PORT> --dbpath ...
=====
➡ Appuie sur [ENTRÉE] une fois le nœud relancé...
🔄 Attente de la synchronisation (Resync)...
Nouvel état du cluster :
[localhost:27017] : SECONDARY
[localhost:27018] : PRIMARY
[localhost:27019] : SECONDARY

--- 7. SCÉNARIO CATASTROPHE (DOUBLE PANNE) ---
Pour qu'un Replica Set fonctionne en écriture, il faut une MAJORITÉ (Quorum).
Avec 3 nœuds, majorité = 2.
Si 2 nœuds tombent, il en reste 1. 1 < 2. Donc le dernier devient READ-ONLY.

🔧 ACTION : Tue un DEUXIÈME nœud (n'importe lequel sauf le dernier survivant).
➡ Appuie sur [ENTRÉE] une fois fait...
Tentative d'écriture (devrait échouer ou timeout)...
✓ Écriture bloquée comme prévu : Collection.insert_one() got an unexpected keyword argument 'write_concern'
Le cluster est passé en mode LECTURE SEULE (Secondary) pour protéger les données.
```

- **Temps de bascule mesuré** : ~0.00 à 2.00 secondes (variable selon la latence réseau locale).
- **Intégrité** : Le document inséré juste avant la panne a été retrouvé sur le nouveau primaire, prouvant que la réplication avait eu lieu.

Scénario 2 : Réintégration (Failback)

Action : Redémarrage du nœud 27017.

- **Résultat** : Le nœud a rejoint le cluster en tant que **SECONDARY** et a synchronisé les données manquées pendant son arrêt. Il n'a pas repris sa place de chef immédiatement (pour la stabilité du cluster).

Scénario 3 : Double Panne (Perte du Quorum)

Action : Arrêt d'un deuxième nœud. Il ne reste qu'un seul serveur en vie (1/3).

- **Observation** :
 - Le dernier nœud passe (ou reste) en état **SECONDARY**.
 - Les tentatives d'écriture échouent (AutoReconnect ou erreur de WriteConcern).
- **Analyse** : C'est le comportement attendu. MongoDB exige une majorité stricte ($N/2 + 1$) pour accepter des écritures. Avec 1 nœud sur 3, la majorité (2) n'est pas atteinte. Le cluster se verrouille en **Lecture Seule** pour éviter les conflits de données (Split Brain).

4. Préparation de l'Application (T3.3)

Afin de préparer l'interface Web (Phase 4), nous avons configuré le projet Django pour qu'il communique avec le cluster.

Configuration (`settings.py`) :

Python

```
MONGO_URI = "mongodb://localhost:27017,localhost:27018,localhost:27019/?  
replicaSet=rs0"
```

L'utilisation de cette URI contenant les trois adresses (Seed List) permet au driver PyMongo de découvrir automatiquement la topologie. Si le primaire change, Django le saura automatiquement sans redémarrage.

5. Conclusion Phase 3

Nous disposons désormais d'une infrastructure de données résiliente.

- **Haute Disponibilité** : Le service survit à la perte d'un serveur.
- **Sécurité des données** : La réplication protège contre la corruption de disque sur un nœud.
- **Évolutivité** : Nous pourrions ajouter des nœuds supplémentaires pour répartir la charge de lecture (`readPreference`).